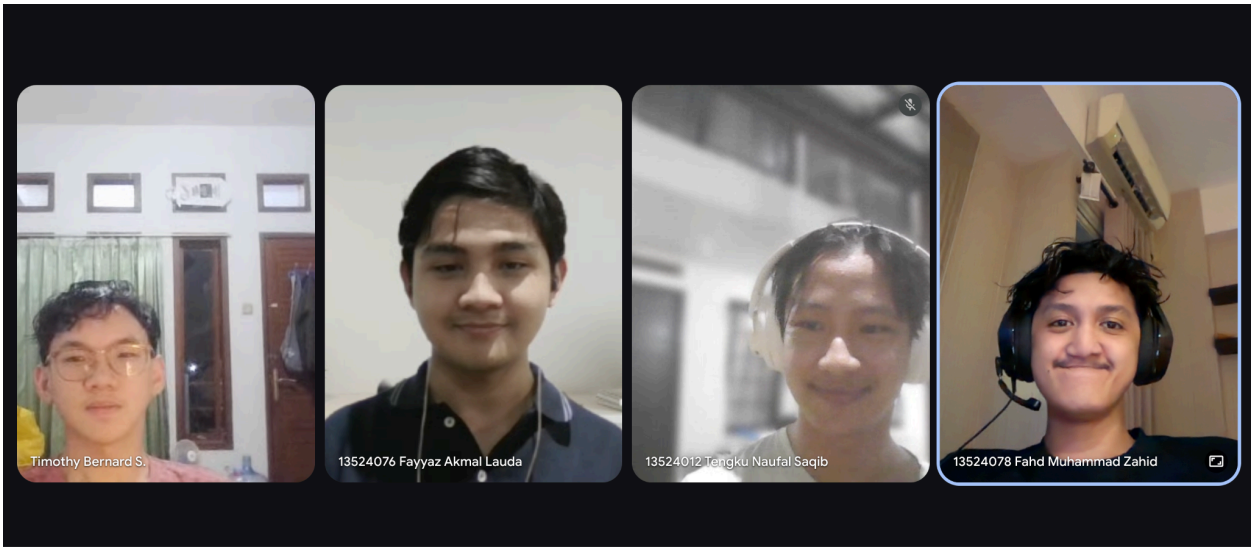
Laporan Tugas Besar IF2224 Teori Bahasa Formal dan Otomata
Milestone 1 : Implementasi Lexical Analyzer pada Arion Interpreter Menggunakan Deterministic Finite Automata (DFA)



Disusun Oleh Kelompok :

Fahd Muhammad Zahid	13524078
Timothy Bernard Soeharto	13524092
Fayyaz Akmal Lauda	13524076
Tengku Naufal Saqib	13524012

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG 2024

BAB I
Teori Singkat

1.1 Lexical Analyzer

Lexical analyzer adalah tahap awal dalam proses kompilasi atau interpretasi bahasa pemrograman. Pada tahap ini, kode sumber yang masih berupa rangkaian karakter diubah menjadi token-token yang memiliki makna. Proses ini bertujuan untuk memudahkan tahap parsing, karena parser tidak lagi membaca karakter mentah, melainkan token yang sudah terstruktur.

Lexical analyzer dijalankan oleh komponen yang disebut lexer. Lexer membaca kode dari kiri ke kanan, mengenali pola tertentu, lalu menghasilkan token sesuai jenisnya. Selain itu, lexer juga mengabaikan spasi, tab, dan komentar, serta dapat mendeteksi karakter yang tidak dikenali. Proses ini umumnya diimplementasikan menggunakan Deterministic Finite Automata (DFA).

1.2 Token

Token adalah unit terkecil dalam program yang masih memiliki arti. Token merupakan hasil dari proses analisis leksikal dan digunakan sebagai input untuk tahap parsing.

Contoh token antara lain keyword, identifier, operator, angka, dan tanda baca. Sebuah token biasanya terdiri dari jenis token dan nilainya. Misalnya: IDENT (a), BECOMES, dan INTCON (5). Dengan adanya token, struktur program menjadi lebih mudah dipahami dan diproses oleh compiler atau interpreter.

1.3 Deterministic Finite Automata (DFA)

Deterministic Finite Automata (DFA) merupakan model matematika yang digunakan dalam mengenali bahasa reguler melalui sekumpulan status (*state*) dan transisi yang terdefinisi secara pasti. Karakteristik utama dari DFA adalah sifat deterministiknya, di mana untuk setiap status dan simbol masukan tertentu, tepat hanya terdapat satu transisi tunggal yang menuju ke status berikutnya. Secara formal, sebuah DFA didefinisikan dalam lima elemen utama (*5-tuple*), yakni himpunan status berhingga (Q), alfabet masukkan (Σ), fungsi transisi (δ), status awal (q_0), dan himpunan status akhir atau *accepting states (F)*.

Dalam pengerjaan *Lexical Analyzer* Arion ini, DFA diimplementasikan sebagai mesin pengenal pola yang bertugas mengelompokkan aliran karakter mentah menjadi unit-unit leksikal yang disebut *token*. Proses ini dilakukan dengan membaca karakter masukan satu demi satu (*char-by-char*) dan berpindah antar status berdasarkan aturan transisi yang telah dirancang dalam diagram. Ketika mesin mencapai status akhir (*final state*) dan tidak ada transisi valid lebih lanjut, urutan karakter tersebut akan diidentifikasi sebagai jenis *token* tertentu, seperti *keyword*, *identifier*, atau *operator*. Namun, jika mesin berhenti di status yang bukan merupakan status akhir atau menerima masukan yang tidak memiliki transisi valid, maka program akan melaporkannya sebagai kesalahan leksikal (*lexical error*).

1.4 Data Flow Diagram (DFD)

Data Flow Diagram (DFD) merupakan suatu model logika fungsional yang digunakan untuk memvisualisasikan bagaimana aliran data bergerak dan ditransformasikan di dalam sebuah sistem. DFD menggambarkan secara terstruktur mengenai asal masukan data (input), proses fungsional yang mengubah data tersebut, tempat penyimpanan data (data store), serta tujuan akhir dari data keluaran (output) tanpa terikat pada implementasi perangkat keras. Pendekatan grafis ini mempermudah pengembang untuk memetakan spesifikasi dan arsitektur sistem secara komprehensif, mulai dari tingkat gambaran umum (Diagram Konteks) hingga rincian proses teknis di bawahnya.

Dalam perancangan compiler pada tahap ini, DFD digunakan untuk memvisualisasikan cara kerja lexical analyzer. Input berupa file *source code* dimasukkan ke dalam proses lexer. Di dalam proses ini, data dievaluasi berdasarkan aturan transisi DFA yang telah dirancang. Output atau hasil akhir dari aliran data ini adalah daftar token (token stream) yang terformat dan pesan error (apabila ditemukan karakter yang tidak memenuhi aturan DFA).

BAB II
Perancangan Dan Implementasi

2.1 Perancangan Diagram Transisi DFA

Bagian ini menjelaskan logika perancangan mesin *Deterministic Finite Automata (DFA)* yang digunakan untuk mengenali pola-pola bahasa Arion. Untuk memperjelas alur perpindahan status pada mesin DFA, berikut adalah tabel deskripsi fungsi masing-masing state dan transisinya :

State	Input Karakter -> Next State	Next State	Deskripsi Action
0	A | a	a	State awal; baca 'a' (huruf 'a'/'A') → masuk state a untuk jalur keyword
a	R | r	ar	Di state a; baca 'r' → masuk state ar
ar	R | r	arr	Di state ar; baca 'r' → masuk state arr
arr	A | a	arra	Di state arr; baca 'a' → masuk state arra
arra	Y | y	array	Di state arra; baca 'y' → masuk state array (accepting: ARRAYSY)
arraysy	newline | space	0	ARRAYSY dikenali; emit token ARRAYSY, kembali ke state 0

a	N | n	an	Di state a; baca 'n' → masuk state an
an	D | d	and	Di state an; baca 'd' → masuk state and (accepting: ANDSY)
andsy	newline | space	0	ANDSY dikenali; emit token ANDSY, kembali ke state 0
0	B | b	b	State awal; baca 'b' → masuk state b
b	E | e	be	Di state b; baca 'e' → masuk state be
be	G | g	beg	Di state be; baca 'g' → masuk state beg
beg	I | i	begi	Di state beg; baca 'i' → masuk state begi
begi	N | n	begin	Di state begi; baca 'n' → masuk state begin (accepting: BEGINSY)
beginsy	newline | space	0	BEGINSY dikenali; emit token BEGINSY, kembali ke state 0

c	A | a	ca	Di state c; baca 'a' → masuk state ca
ca	S | s	cas	Di state ca; baca 's' → masuk state cas
cas	E | e	case	Di state cas; baca 'e' → masuk state case (accepting: CASESY)
casesy	newline | space	0	CASESY dikenali; emit token CASESY, kembali ke state 0
0	C | c	c	State awal; baca 'c' → masuk state c
c	O | o	co	Di state c; baca 'o' → masuk state co
co	N | n	con	Di state co; baca 'n' → masuk state con
con	S | s	cont	Di state con; baca 's' → masuk state cont (menuju constsy)
contsy	newline | space	0	CONSTSY dikenali; emit token CONSTSY, kembali ke state 0

0	D | d	d	State awal; baca 'd' → masuk state d
d	I | i	di	Di state d; baca 'i' → masuk state di
di	V | v	div	Di state di; baca 'v' → masuk state div (accepting: IDIV)
idiv	newline | space	0	IDIV dikenali; emit token IDIV, kembali ke state 0
d	O | o	do	Di state d; baca 'o' → masuk state do
do	newline | space	dosy	Di state do; delimiter → accepting DOSY
dosy	newline | space	0	DOSY dikenali; emit token DOSY, kembali ke state 0
do	W | w	dow	Di state do; baca 'w' → masuk state dow
dow	N | n	down	Di state dow; baca 'n' → masuk state down
down	T | t	downt	Di state down; baca 't'

			→ masuk state downt
downt	O | o	downto	Di state downt; baca 'o' → masuk state downto (accepting: DOWNTOSY)
downtosy	newline | space	0	DOWNTOSY dikenali; emit token DOWNTOSY, kembali ke state 0
0	E | e	e	State awal; baca 'e' → masuk state e
e	L | l	el	Di state e; baca 'l' → masuk state el
el	S | s	els	Di state el; baca 's' → masuk state els
els	E | e	else	Di state els; baca 'e' → masuk state else (accepting: ELSESY)
elsesy	newline | space	0	ELSESY dikenali; emit token ELSESY, kembali ke state 0
e	N | n	en	Di state e; baca 'n' → masuk state en

en	D | d	end	Di state en; baca 'd' → masuk state end (accepting: ENDSY)
end	newline | space	endsy	Di state end; delimiter → accepting ENDSY
endsy	newline | space	0	ENDSY dikenali; emit token ENDSY, kembali ke state 0
0	F | f	f	State awal; baca 'f' → masuk state f
f	O | o	fo	Di state f; baca 'o' → masuk state fo
fo	R | r	for	Di state fo; baca 'r' → masuk state for (accepting: FORSY)
forsy	newline | space	0	FORSY dikenali; emit token FORSY, kembali ke state 0
f	U | u	fu	Di state f; baca 'u' → masuk state fu
fu	N | n	fun	Di state fu; baca 'n' → masuk state fun

fun	C | c	func	Di state fun; baca 'c' → masuk state func
func	T | t	funct	Di state func; baca 't' → masuk state funct
funct	I | i	functi	Di state funct; baca 'i' → masuk state functi
functi	O | o	functio	Di state functi; baca 'o' → masuk state functio
functio	N | n	function	Di state functio; baca 'n' → masuk state function (accepting: FUNCTIONSY)
functionsy	newline | space	0	FUNCTIONSY dikenali; emit token FUNCTIONSY, kembali ke state 0
0	I | i	i	State awal; baca 'i' → masuk state i
i	F | f	if	Di state i; baca 'f' → masuk state if (accepting: IFSY)
ifsy	newline | space	0	IFSY dikenali; emit token IFSY, kembali

			ke state 0
0	M | m	m	State awal; baca 'm' → masuk state m
m	O | o	mo	Di state m; baca 'o' → masuk state mo
mo	D | d	mod	Di state mo; baca 'd' → masuk state mod (accepting: IMOD)
imod	newline | space	0	IMOD dikenali; emit token IMOD, kembali ke state 0
0	N | n	n	State awal; baca 'n' → masuk state n
n	O | o	no	Di state n; baca 'o' → masuk state no
no	T | t	not	Di state no; baca 't' → masuk state not (accepting: NOTSY)
notsy	newline | space	0	NOTSY dikenali; emit token NOTSY, kembali ke state 0
0	O | o	o	State awal; baca 'o' → masuk state o

o	F | f	of	Di state o; baca 'f' → masuk state of (accepting: OFSY)
ofsy	newline | space	0	OFSY dikenali; emit token OFSY, kembali ke state 0
o	R | r	or	Di state o; baca 'r' → masuk state or (accepting: ORSY)
orsy	newline | space	0	ORSY dikenali; emit token ORSY, kembali ke state 0
0	P | p	p	State awal; baca 'p' → masuk state p
p	R | r	pr	Di state p; baca 'r' → masuk state pr
pr	O | o	pro	Di state pr; baca 'o' → masuk state pro
pro	C | c	proc	Di state pro; baca 'c' → masuk state proc
proc	E | e	proce	Di state proc; baca 'e' → masuk state proce

proce	D | d	proced	Di state proce; baca 'd' → masuk state proced
proced	U | u	procedu	Di state proced; baca 'u' → masuk state procedu
procedu	R | r	procedur	Di state procedu; baca 'r' → masuk state procedur
procedur	E | e	procedure	Di state procedur; baca 'e' → masuk state procedure (accepting: PROCEDURESY)
proceduresy	newline | space	0	PROCEDURESY dikenali; emit token PROCEDURESY, kembali ke state 0
pro	G | g	prog	Di state pro; baca 'g' → masuk state prog
prog	R | r	progr	Di state prog; baca 'r' → masuk state progr
progr	A | a	progra	Di state progr; baca 'a' → masuk state progra
progra	M | m	program	Di state progra; baca 'm' → masuk state

			program (accepting: PROGRAMSY)
programsy	newline | space	0	PROGRAMSY dikenali; emit token PROGRAMSY, kembali ke state 0
0	R | r	r	State awal; baca 'r' → masuk state r
r	E | e	re	Di state r; baca 'e' → masuk state re
re	P | p	rep	Di state re; baca 'p' → masuk state rep
rep	E | e	repe	Di state rep; baca 'e' → masuk state repe
repe	A | a	repea	Di state repe; baca 'a' → masuk state repea
repea	T | t	repeat	Di state repea; baca 't' → masuk state repeat (accepting: REPEATSY)
repeatsy	newline | space	0	REPEATSY dikenali; emit token REPEATSY, kembali ke state 0

re	C | c	rec	Di state re; baca 'c' → masuk state rec
rec	O | o	reco	Di state rec; baca 'o' → masuk state reco
reco	R | r	recor	Di state reco; baca 'r' → masuk state recor
recor	D | d	record	Di state recor; baca 'd' → masuk state record (accepting: RECORDSY)
recordsy	newline | space	0	RECORDSY dikenali; emit token RECORDSY, kembali ke state 0
0	T | t	t	State awal; baca 't' → masuk state t
t	H | h	th	Di state t; baca 'h' → masuk state th
th	E | e	the	Di state th; baca 'e' → masuk state the
the	N | n	then	Di state the; baca 'n' → masuk state then (accepting: THENSY)

thensy	newline | space	0	THENSY dikenali; emit token THENSY, kembali ke state 0
t	O | o	to	Di state t; baca 'o' → masuk state to (accepting: TOSY)
tosy	newline | space	0	TOSY dikenali; emit token TOSY, kembali ke state 0
t	Y | y	ty	Di state t; baca 'y' → masuk state ty
ty	P | p	typ	Di state ty; baca 'p' → masuk state typ
typ	E | e	type	Di state typ; baca 'e' → masuk state type (accepting: TYPESY)
typesy	newline | space	0	TYPESY dikenali; emit token TYPESY, kembali ke state 0
0	U | u	u	State awal; baca 'u' → masuk state u
u	N | n	un	Di state u; baca 'n' → masuk state un

un	T | t	unt	Di state un; baca 't' → masuk state unt
unt	I | i	unti	Di state unt; baca 'i' → masuk state unti
unti	L | l	until	Di state unti; baca 'l' → masuk state until (accepting: UNTILSY)
untilsy	newline | space	0	UNTILSY dikenali; emit token UNTILSY, kembali ke state 0
0	V | v	v	State awal; baca 'v' → masuk state v
v	A | a	va	Di state v; baca 'a' → masuk state va
va	R | r	var	Di state va; baca 'r' → masuk state var (accepting: VARSY)
varsy	newline | space	0	VARSY dikenali; emit token VARSY, kembali ke state 0
0	W | w	w	State awal; baca 'w' → masuk state w

w	H | h	wh	Di state w; baca 'h' → masuk state wh
wh	I | i	whi	Di state wh; baca 'i' → masuk state whi
whi	L | l	whil	Di state whi; baca 'l' → masuk state whil
whil	E | e	while	Di state whil; baca 'e' → masuk state while (accepting: WHILESY)
whilesy	newline | space	0	WHILESY dikenali; emit token WHILESY, kembali ke state 0
ident	[a-z] | [A-Z] | 0-9	l1	State awal identifikasi token identifier/keyword. Dari q0, baca satu huruf pertama → masuk l1
l1	[a-z] | [A-Z] | [0-9] (alnum) → l1 (self-loop) newline | space | delimiter → l2	l1 (loop) | l2	Akumulasi karakter alfanumerik (self-loop). Saat delimiter/spasi/newline ditemui → keluar ke l2

l2	—	l3	Transisi dari l1 setelah delimiter terdeteksi. Lexeme lengkap terkumpul → teruskan ke l3
l3	— (cek keyword map)	l4 (accepting keyword) | l4 (accepting ident)	Cek lexeme (case-insensitive) ke keyword map. Jika cocok → emit token keyword; jika tidak → emit IDENT
l4	—	0	State accepting. Emit token keyword atau IDENT dengan value lexeme, kembali ke state 0
charcon	— (kondisi: panjang isi = 1 karakter)	0 (accepting)	Kutip penutup ditemukan setelah tepat 1 karakter terbaca → emit token CHARCON (nilai = karakter tunggal tersebut)
string	— (kondisi: panjang isi = 0 atau > 1 karakter)	0 (accepting)	Kutip penutup ditemukan setelah 0 atau lebih dari 1 karakter → emit token STRING (nilai = isi string)
intcon	— (kondisi: '.' tidak diikuti digit, atau	0 (accepting)	State accepting integer. Tidak ditemukan titik

	tidak ada '.')		desimal yang valid → emit INTCON, kembali ke state 0
n1	[0-9] → n1 (self-loop) '.' diikuti [0-9] → n2 '.' diikuti bukan digit → emit INTCON (titik tidak dikonsumsi)	n1 (loop) | n2 | 0 (accepting intcon)	Akumulasi digit integer (self-loop). Jika '.' diikuti digit → masuk jalur realcon (n2). Jika '.' tidak diikuti digit → emit INTCON sekarang, biarkan '.' untuk token berikutnya (PERIOD)
0	— (kondisi: n2 selesai membaca digit desimal)	realcon	State accepting real. Digit desimal telah terkumpul di n2 → emit REALCON (misal 3.14), kembali ke state 0
0	, (koma)	comma	Baca karakter ','; langsung emit token COMMA, kembali ke state 0
0	+ (tambah)	plus	Baca karakter '+'; langsung emit token PLUS, kembali ke state 0
0	- (kurang)	minus	Baca karakter '-'; langsung emit token MINUS, kembali ke state 0

0	* (bintang)	times	Baca karakter '*'; langsung emit token TIMES, kembali ke state 0
0	/ (garis miring)	rdiv	Baca karakter '/'; langsung emit token RDIV, kembali ke state 0
0	> diikuti bukan '='	gtr	Baca '>'; jika karakter berikutnya bukan '=' → emit GTR (>), kembali ke state 0
0	= (setelah membaca '>')	gep	Intermediate state setelah '>'. Baca '=' → emit GEQ (>=), kembali ke state 0
0	> (setelah membaca '<')	neq	Intermediate state setelah '<'. Baca '>' → emit NEQ (<>), kembali ke state 0
0	< diikuti bukan '=' dan bukan '>'	lss	Baca '<'; jika bukan '=' dan bukan '>' → emit LSS (<), kembali ke state 0
0	= (setelah membaca '<')	leq	Intermediate state setelah '<'. Baca '=' → emit LEQ (<=), kembali ke state 0

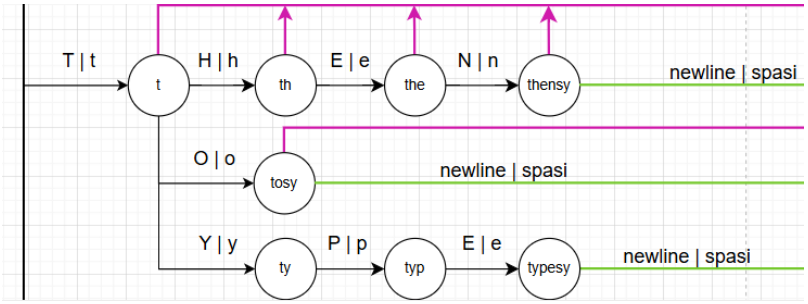
e1	= (karakter '=' pertama)	eql | UNKNOWN	Intermediate state: karakter '=' pertama dibaca. Jika '=' berikutnya → lanjut ke eql; jika bukan '=' → emit UNKNOWN (single '=' tidak valid di Arion)
unknown		0	
0	= (karakter '=' kedua)	eql	Baca '=' kedua setelah e1; emit token EQL (==), kembali ke state 0
0	: diikuti bukan '='	colon	Baca ':'; jika bukan '=' → emit COLON (:), kembali ke state 0
0	= (setelah membaca ':')	becomes	Intermediate state setelah ':'. Baca '=' → emit BECOMES (:=), kembali ke state 0
0	; (titik koma)	semicolon	Baca karakter ';'; langsung emit token SEMICOLON, kembali ke state 0
0	. diikuti bukan digit	period	Baca '.'; jika bukan digit → emit PERIOD. (Jika diikuti digit, ditangani di n1 sebagai awal bagian

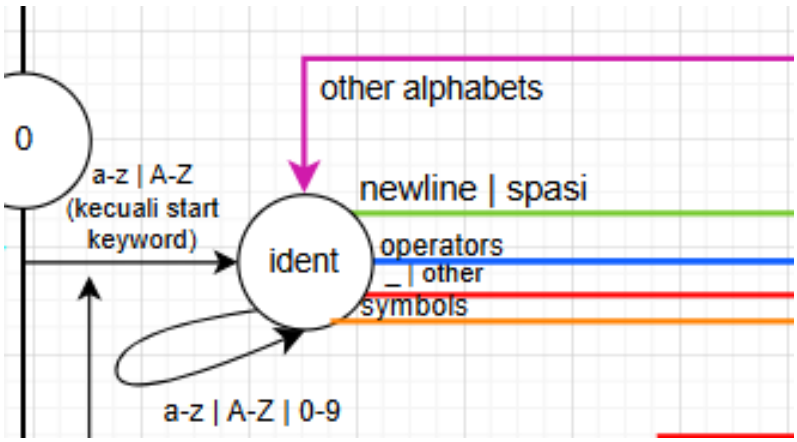
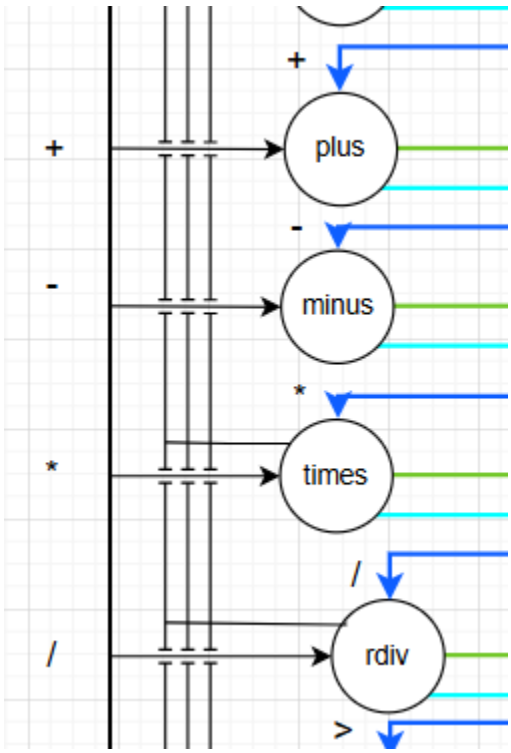
			desimal realcon)
0	[(kurung siku buka)	lbrack	Baca karakter '['; langsung emit token LBRACK, kembali ke state 0
0	] (kurung siku tutup)	rbrack	Baca karakter ']'; langsung emit token RBRACK, kembali ke state 0
0	(diikuti bukan '*'	lparent	Baca '('; jika bukan '*' → emit LPARENT. (Jika diikuti '*', ditangani di c2 sebagai awal komentar (* *))
0	) (kurung tutup)	rparent	Baca karakter ')'; langsung emit token RPARENT, kembali ke state 0
c1	{ (kurung kurawal buka)	cc1	Baca '{' dari state 0; masuk ke state cc1 (badan komentar tipe { })
c2	* (setelah membaca '(')	cc3	Dari state 0 baca '(' lalu jika setelahnya '*'; konsumsi '*' → masuk cc3 (badan komentar tipe (* *))

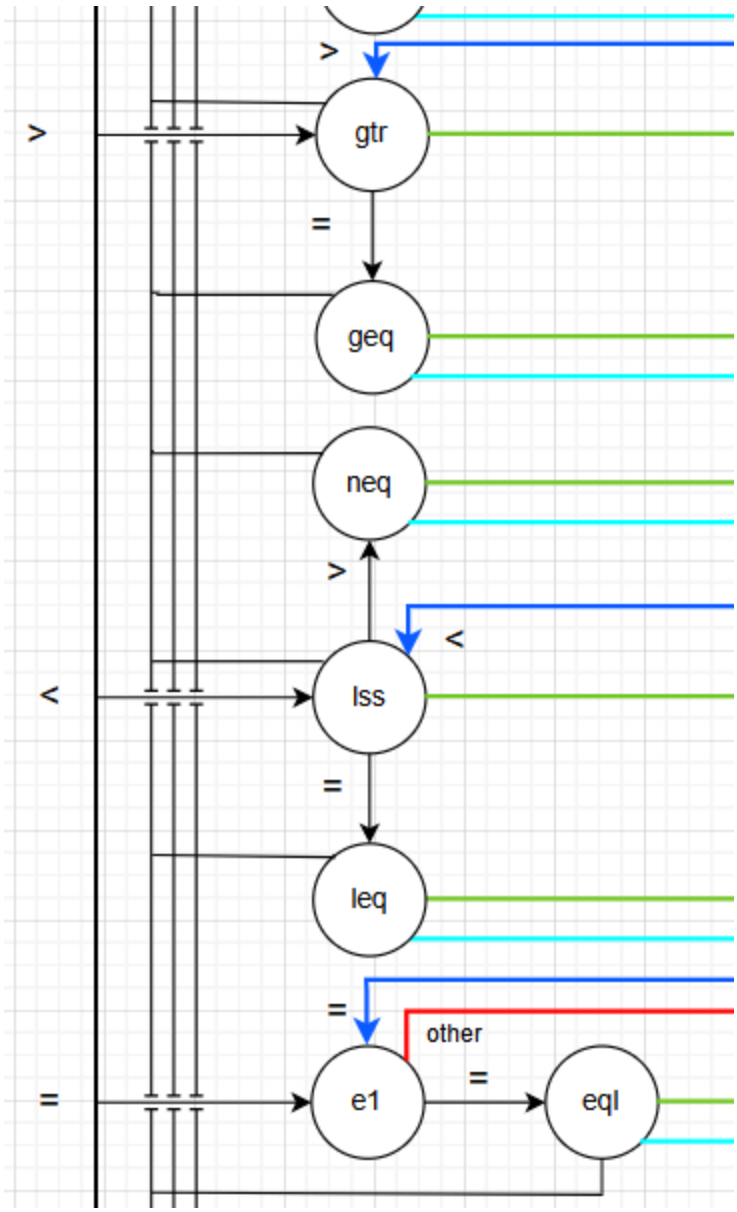
cc1	[bukan '}' dan bukan '{'] → cc1 (self-loop) } → cc2 (closing) { atau (* atau EOF → UNKNOWN	cc1 (loop) | cc2 | UNKNOWN	Self-loop di dalam badan komentar { }. Setiap karakter selain '}' dikumpulkan. Jika '}' → cc2. Jika '{' nested atau EOF sebelum tutup → emit UNKNOWN (error)
cc2	} (kurung kurawal tutup)	0 (accepting)	Karakter '}' ditemukan; komentar { } selesai → emit token COMMENT dengan isi badan komentar, kembali ke state 0
cc3	[bukan '*'] → cc3 (self-loop) '*' diikuti ')' → cc4 (closing) '*' diikuti bukan ')' → cc3 { atau } → UNKNOWN EOF → UNKNOWN	cc3 (loop) | cc4 | UNKNOWN	Self-loop di dalam badan komentar (* *). Karakter selain '*' dikumpulkan. Jika '*' → cek setelahnya ')': jika ada → cc4; jika tidak → tetap cc3. Jika EOF sebelum tutup → UNKNOWN
cc4	*) (bintang diikuti kurung tutup)	0 (accepting)	Urutan '*' + ')' ditemukan; komentar (* *) selesai → emit token COMMENT dengan isi badan, kembali ke state 0
comment	—	0	State accepting untuk semua jenis komentar ({ } maupun (* *)). Token COMMENT

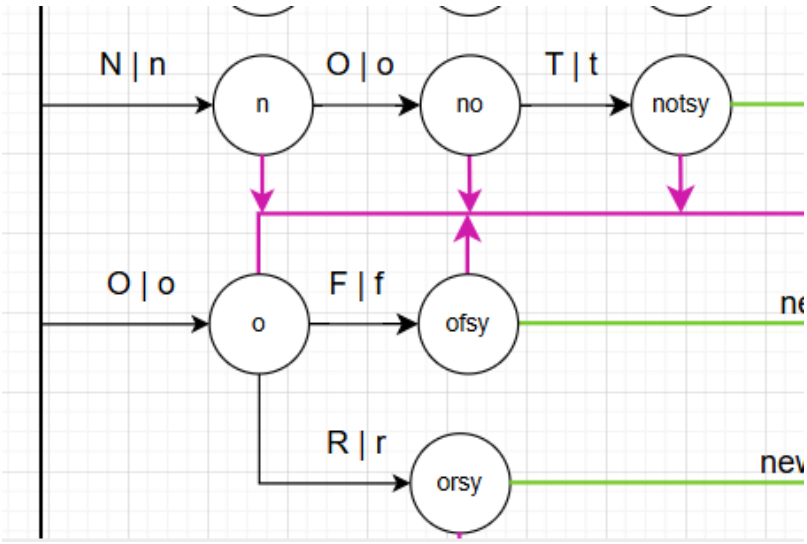
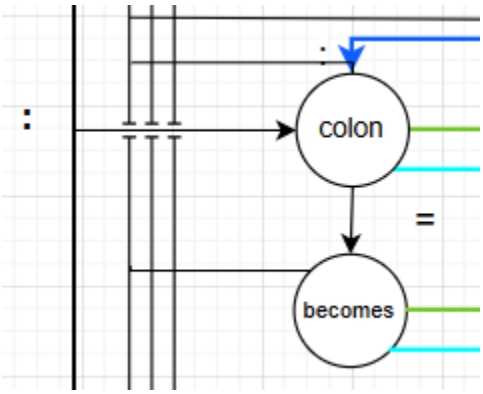
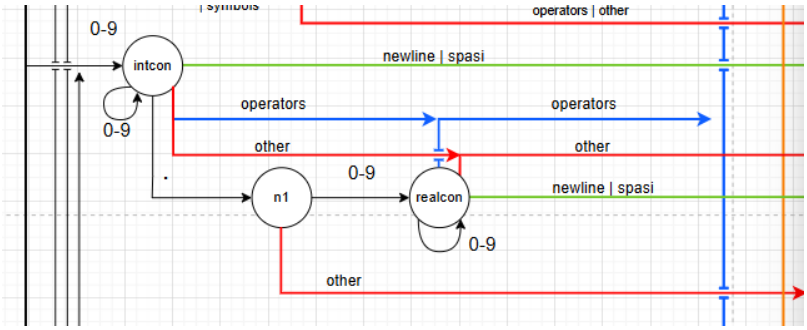
			diterbitkan dengan value = isi badan komentar, lalu kembali ke state 0

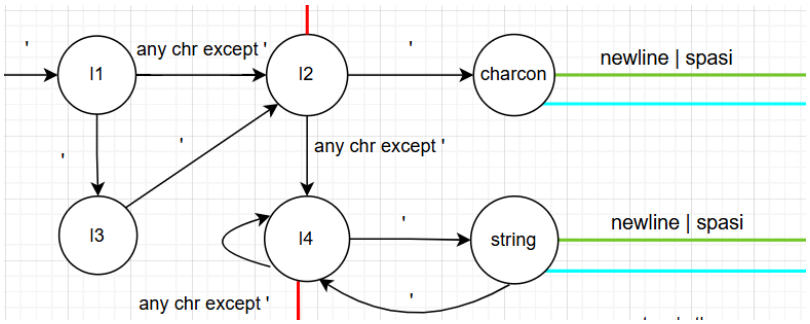
Gambar 3.2 : Diagram Transisi DFA Arion Per Bagian

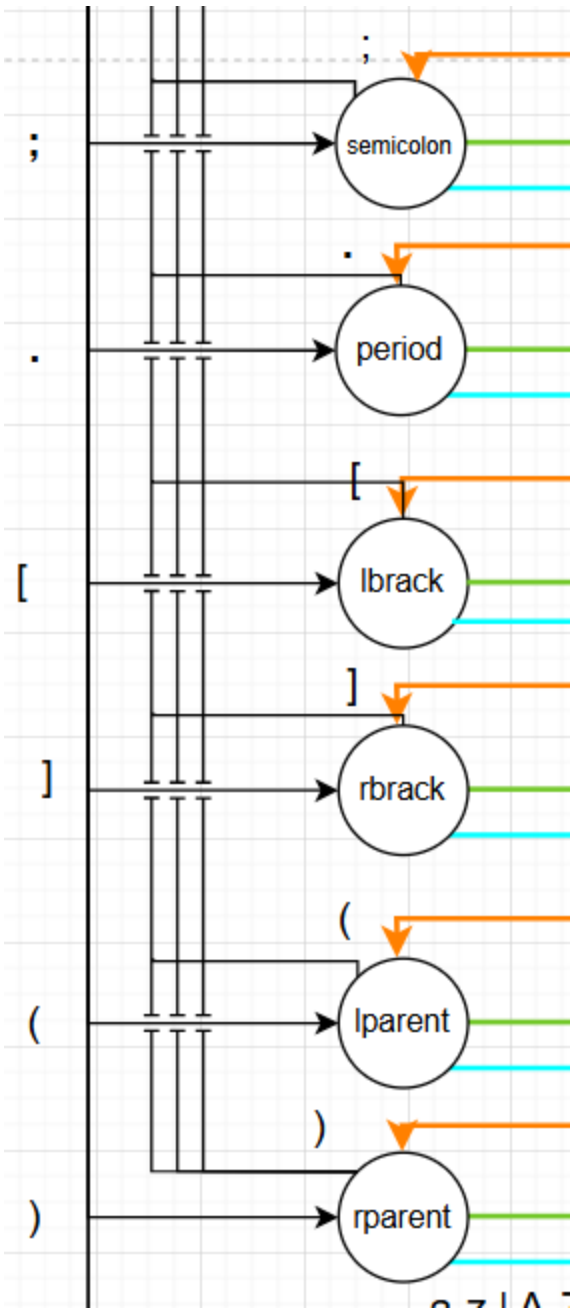
Tipe	Diagram DFA	Penjelasan
KEYWORD		Keyword adalah kata khusus yang memiliki makna tetap dalam bahasa pemrograman dan tidak boleh digunakan sebagai identifier, seperti begin, end, if, while, dan lain-lain. Pada diagram DFA, setiap keyword direpresentasikan sebagai rangkaian state yang membaca karakter satu per satu sesuai urutan hurufnya; jika seluruh rangkaian cocok, maka automata mencapai state akhir (final state) yang menandakan token tersebut valid sebagai keyword. Jika di tengah jalan terjadi ketidaksesuaian karakter, maka proses akan dialihkan ke kemungkinan lain seperti identifier.

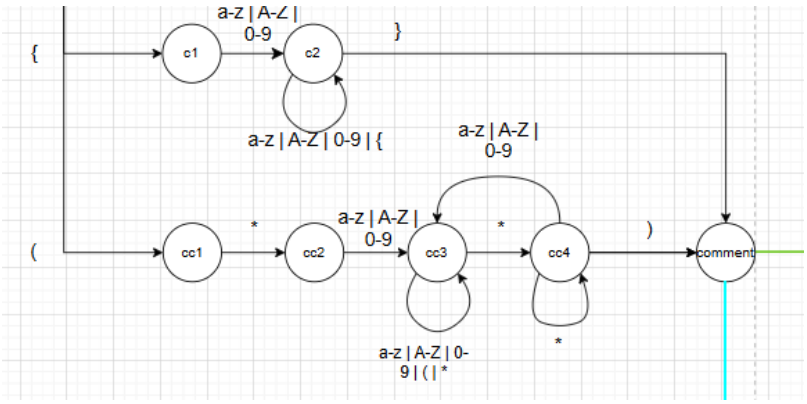
IDENT		Identifier adalah nama yang diberikan untuk variabel, fungsi, atau entitas lain, dengan aturan dimulai dari huruf (a-z atau A-Z) dan dapat diikuti oleh huruf atau angka. Dalam DFA, proses dimulai dari state awal yang menerima huruf, kemudian berpindah ke state yang memungkinkan pengulangan (loop) untuk huruf dan angka, sehingga dapat membentuk identifier dengan panjang bebas. Jika rangkaian karakter tidak cocok dengan keyword tertentu, maka secara default akan diklasifikasikan sebagai identifier.
ARITHMETIC _OPERATOR		Arithmetic operator adalah simbol yang digunakan untuk operasi matematika seperti penjumlahan (+), pengurangan (-), perkalian (*), dan pembagian (/). Pada DFA, operator ini sangat sederhana karena setiap simbol langsung dikenali dan membawa automata dari state awal ke state akhir tanpa perlu rangkaian panjang, sehingga proses identifikasinya bersifat satu langkah.

RELATIONAL_OPERATOR		Relational operator digunakan untuk membandingkan dua nilai, seperti =, <, >, <=, >=, dan <>. Dalam DFA, operator ini dapat terdiri dari satu atau dua karakter, sehingga automata harus mengecek kemungkinan lanjutan setelah membaca karakter pertama (misalnya < bisa menjadi <, <=, atau <>). Jika kombinasi karakter valid, maka akan mencapai state akhir yang sesuai dengan jenis operator relasi tersebut.

LOGICAL_OPERATOR		Logical operator seperti and, or, dan not digunakan untuk operasi logika dalam ekspresi boolean. Pada DFA, operator ini diproses mirip dengan keyword, yaitu dengan membaca huruf satu per satu hingga membentuk kata lengkap. Jika seluruh karakter cocok dengan salah satu operator logika, maka automata mencapai state akhir; jika tidak, kemungkinan akan diklasifikasikan sebagai identifier.
ASSIGN_OPERATOR		Assign operator dalam diagram ini direpresentasikan oleh simbol := yang berarti penugasan nilai ke variabel. DFA memprosesnya dengan membaca karakter : terlebih dahulu, kemudian memeriksa apakah diikuti oleh =. Jika kedua karakter tersebut muncul berurutan, maka automata mencapai state akhir dan token dikenali sebagai operator assignment.
NUMBER		Number mencakup bilangan integer dan real. Integer terdiri dari satu atau lebih digit (0-9), sedangkan bilangan real memiliki bagian desimal yang dipisahkan oleh titik (misalnya 3.14). Dalam DFA, angka pertama akan membawa automata ke state yang memungkinkan

		pengulangan digit, dan jika ditemukan titik (.), maka berpindah ke state lain untuk memproses bagian desimal hingga mencapai state akhir sebagai bilangan real.
CHAR_LITERAL dan STRING_LITERAL		Char literal dan string literal adalah representasi karakter atau kumpulan karakter yang dibatasi oleh tanda kutip tunggal (''). DFA memulai dari karakter pembuka ', kemudian menerima berbagai karakter di dalamnya (huruf, angka, dll.), dan harus ditutup kembali dengan ' untuk mencapai state akhir. Jika tidak ditutup dengan benar, maka tidak dianggap valid.

SYMBOL		Symbol adalah karakter khusus seperti tanda kurung (,), kurung siku [,], titik koma ;, titik ., koma ,, dan lain-lain yang memiliki fungsi sintaks tertentu. Dalam DFA, setiap simbol biasanya langsung dikenali sebagai token valid hanya dengan satu transisi dari state awal ke state akhir karena tidak memerlukan kombinasi karakter.

COMMENT		Comment adalah bagian kode yang diabaikan oleh compiler/interpreter, ditulis dalam bentuk { ... } atau (* ... *). Pada DFA, proses dimulai saat menemukan pembuka komentar ({ atau (*), kemudian automata akan terus membaca semua karakter di dalamnya tanpa memproses sebagai token lain hingga menemukan penutup yang sesuai (} atau *)), yang kemudian membawa ke state akhir sebagai token komentar.

2.2 Implementasi Program Lexical Analyzer

2.3 Implementasi Program

2.3.1 Struktur Program

Program lexical analyzer diimplementasikan dalam bahasa C++ dengan struktur direktori sebagai berikut :

```
.
├── doc
├── README.md
├── src
│   ├── lexer.h
│   ├── main.cpp
│   └── token.h
├── test
│   ├── input-1.txt
│   └── input-2.txt
├── Makefile
└── lexer
```

Penjelasan masing-masing file :

- **token.h** : Mendefinisikan enum TokenType yang mencantumkan seluruh jenis token yang dikenali oleh lexer Arion, seperti PROGRAMSY, IDENT, INTCON,

BECOMES, dan sebagainya. File ini juga mendefinisikan struct Token yang menyimpan pasangan (type, value).

- **lexer.h** : Mendeklarasikan kelas Lexer beserta seluruh metode publiknya, termasuk konstruktor yang menerima stream input dan metode nextToken() yang mengembalikan token berikutnya satu per satu.
- **main.cpp** : Titik masuk program. Membuka file input yang diberikan sebagai argumen command-line, membuat instance Lexer, lalu memanggil nextToken() dalam loop sampai token EOF diterima dan mencetak setiap token ke stdout

2.3.2 Logika DFA

Mesin DFA diimplementasikan di dalam metode nextToken() pada kelas Lexer menggunakan pendekatan explicit state variable. Setiap pemanggilan nextToken() memulai ulang mesin dari state 0 (state awal) dan membaca karakter satu per satu dari buffer input.

Alur kerja utama :

- Whitespace dan newline diabaikan secara eksplisit di awal setiap iterasi sebelum mesin berpindah ke state yang relevan.
- Karakter pertama yang dibaca menentukan cabang DFA yang akan dimasuki: huruf menuju jalur keyword/identifier, digit menuju jalur number, tanda kutip tunggal menuju jalur char/string literal, dan simbol-simbol operator serta tanda baca ditangani dengan transisi satu langkah.
- Untuk keyword dan identifier, karakter alfanumerik diakumulasikan ke dalam sebuah string lexeme menggunakan self-loop pada state l1. Setelah delimiter terdeteksi, lexeme diperiksa terhadap keyword map (case-insensitive) menggunakan std::map<std::string, TokenType>. Jika cocok, token keyword diterbitkan; jika tidak, token IDENT diterbitkan dengan value berisi lexeme aslinya.
- Untuk bilangan, digit diakumulasikan pada state n1. Jika karakter titik (.) ditemukan dan karakter berikutnya adalah digit, mesin berpindah ke state n2 untuk memproses bagian desimal dan menghasilkan token REALCON. Jika titik tidak diikuti digit, token INTCON diterbitkan dan titik dikembalikan ke buffer (unget) agar diproses sebagai PERIOD.
- Untuk komentar, setelah menemukan pembuka { atau (*, mesin masuk ke state cc1 atau cc3 yang melakukan self-loop atas semua karakter hingga penutup yang sesuai (} atau *)) ditemukan.

2.3.3 Penanganan Error

Lexer dirancang untuk bersifat resilient, artinya ia tidak langsung berhenti saat menemukan karakter yang tidak valid. Strategi penanganan error yang diterapkan adalah sebagai berikut:

- **Karakter Tidak Dikenal (Unknown Token):** Setiap karakter yang tidak memiliki transisi valid dari state 0 (misalnya @, #, $) akan menghasilkan token UNKNOWN dengan value berupa karakter tersebut. Lexer kemudian melanjutkan proses tokenisasi dari state 0 untuk karakter berikutnya, sehingga sisa program tetap dapat diproses.
- **Komentar Tidak Ditutup (Unterminated Comment):** Apabila EOF ditemukan sebelum penutup komentar (} atau *)) ditemukan, mesin akan keluar dari state cc1/cc3 dan menerbitkan token UNKNOWN untuk menandai komentar yang tidak valid tersebut.
- **Single Equals (=):** Karakter = tunggal (bukan bagian dari ==, :=, >=, atau <=) tidak valid dalam bahasa Arion. State e1 akan memeriksa karakter berikutnya; jika bukan = maka token UNKNOWN diterbitkan untuk karakter = tersebut.
- **Komentar Nested Tidak Valid:** Pendeteksian tanda { di dalam badan komentar { } (state cc1) akan menyebabkan token UNKNOWN diterbitkan karena bahasa Arion tidak mendukung komentar bersarang.

BAB III
Pengujian

3.1 Hasil Pengujian

3.1.1 Metodologi Pengujian

Pengujian lexical analyzer ini dilakukan menggunakan metode black-box testing dan pengujian fungsional unit. Masukan (input) berupa potongan kode sumber simulasi dalam bahasa Arion yang dirancang untuk mengevaluasi berbagai alur pada diagram transisi DFA. Evaluasi dilakukan dengan membandingkan daftar token keluaran (output token stream) yang dihasilkan oleh program dengan ekspektasi matematis dari spesifikasi. Parameter kelulusan pengujian mencakup keakuratan identifikasi token, keberhasilan membuang karakter whitespace dan komentar, resolusi konflik keyword dan identifier, serta ketepatan dalam melaporkan lexical error.

3.1.2 Hasil Pengujian

1.2.1. Test Case 1

Tujuan	Menguji pengenalan token-token dasar program

Input	`program Halo;`
	`var`
	`  x: integer;`
	`begin`
	`  x := 42;`
	`end.`

Expected Output	`programsy`
	`ident (Halo)`
	`semicolon`
	`varsy`
	`ident (x)`
	`colon`
	`ident (integer)`
	`semicolon`
	`beginsy`
	`ident (x)`
	`becomes`
	`intcon (42)`
	`semicolon`
	`endsy`
	`period`

Output	programsy ident (Halo) semicolon varsy ident (x) colon ident (integer) semicolon beginsy ident (x) becomes intcon (42) semicolon endsy period
Bukti	1/TBS-Tubes-IF2224-2026$ ./lexer test/input-1.txt programsy ident (Halo) semicolon varsy ident (x) colon ident (integer) semicolon beginsy ident (x) becomes intcon (42) semicolon endsy period

1.2.2. Test Case 2

Tujuan	Menguji pengenalan seluruh operator aritmatika

Input	program OprTest;
	var
	  a, b, c: integer;
	begin
	  a := 10 + 3;
	  b := 10 - 3;

	`  c := 10 * 3;`
	`  a := 10 div 3;`
	`  b := 10 / 3;`
	`  c := 10 MOD 3;`
	`  if a == b then`
	`    a := 1;`
	`  if a <> b then`
	`    a := 2;`
	`  if a > b then`
	`    a := 3;`
	`  if a >= b then`
	`    a := 4;`
	`  if a < b then`
	`    a := 5;`
	`  if a <= b then`
	`    a := 6;`
	`end.`

Expected Output	`programsy`
	`ident (OprTest)`
	`semicolon`
	`varsy`
	`ident (a)`
	`comma`
	`ident (b)`
	`comma`
	`ident (c)`
	`colon`
	`ident (integer)`
	`semicolon`
	`beginsy`
	`ident (a)`
	`becomes`
	`intcon (10)`
	`plus`
	`intcon (3)`
	`semicolon`
	`ident (b)`
	`becomes`
	`intcon (10)`

	minus intcon (3) semicolon ident (c) becomes intcon (10) times intcon (3) semicolon ident (a) becomes intcon (10) idiv intcon (3) semicolon ident (b) becomes intcon (10) rdiv intcon (3) semicolon ident (c) becomes intcon (10) imod intcon (3) semicolon ifsy ident (a) eql ident (b) thensy ident (a) becomes intcon (1) semicolon ifsy ident (a) neq ident (b)

	thensy ident (a) becomes intcon (2) semicolon ifsy ident (a) gtr ident (b) thensy ident (a) becomes intcon (3) semicolon ifsy ident (a) geq ident (b) thensy ident (a) becomes intcon (4) semicolon ifsy ident (a) lss ident (b) thensy ident (a) becomes intcon (5) semicolon ifsy ident (a) leq ident (b) thensy ident (a) becomes intcon (6)

	semicolon endsy period
Output	programsy ident (OprTest) semicolon varsy ident (a) comma ident (b) comma ident (c) colon ident (integer) semicolon beginsy ident (a) becomes intcon (10) plus intcon (3) semicolon ident (b) becomes intcon (10) minus intcon (3) semicolon ident (c) becomes intcon (10) times intcon (3) semicolon ident (a) becomes intcon (10) idiv intcon (3)

	semicolon ident (b) becomes intcon (10) rdiv intcon (3) semicolon ident (c) becomes intcon (10) imod intcon (3) semicolon ifsy ident (a) eql ident (b) thensy ident (a) becomes intcon (1) semicolon ifsy ident (a) neq ident (b) thensy ident (a) becomes intcon (2) semicolon ifsy ident (a) gtr ident (b) thensy ident (a) becomes intcon (3) semicolon

	ifsy ident (a) geq ident (b) thensy ident (a) becomes intcon (4) semicolon ifsy ident (a) lss ident (b) thensy ident (a) becomes intcon (5) semicolon ifsy ident (a) leq ident (b) thensy ident (a) becomes intcon (6) semicolon endsy period

Bukti	1/TBS-Tubes-IF2224-2026$ ./lexer test/input-2.txt programsy ident (OprTest) semicolon varsy ident (a) comma ident (b) comma ident (c) colon ident (integer) semicolon beginsy ident (a) becomes intcon (10) plus intcon (3) semicolon ident (b) becomes intcon (10) minus intcon (3) semicolon ident (c) becomes intcon (10) times intcon (3)

	semicolon ident (a) becomes intcon (10) idiv intcon (3) semicolon ident (b) becomes intcon (10) rdiv intcon (3) semicolon ident (c) becomes intcon (10) imod intcon (3) semicolon ifsy ident (a) eql ident (b) thensy ident (a) becomes intcon (1) semicolon ifsy ident (a) neq ident (b) thensy

	``` ident (a) becomes intcon (2) semicolon ifsy ident (a) gtr ident (b) thensy ident (a) becomes intcon (3) semicolon ifsy ident (a) geq ident (b) thensy ident (a) becomes intcon (4) semicolon ifsy ident (a) lss ident (b) thensy ident (a) becomes intcon (5) semicolon ```

	ifsy ident (a) leq ident (b) thensy ident (a) becomes intcon (6) semicolon endsy period

1.2.3. Test Case 3

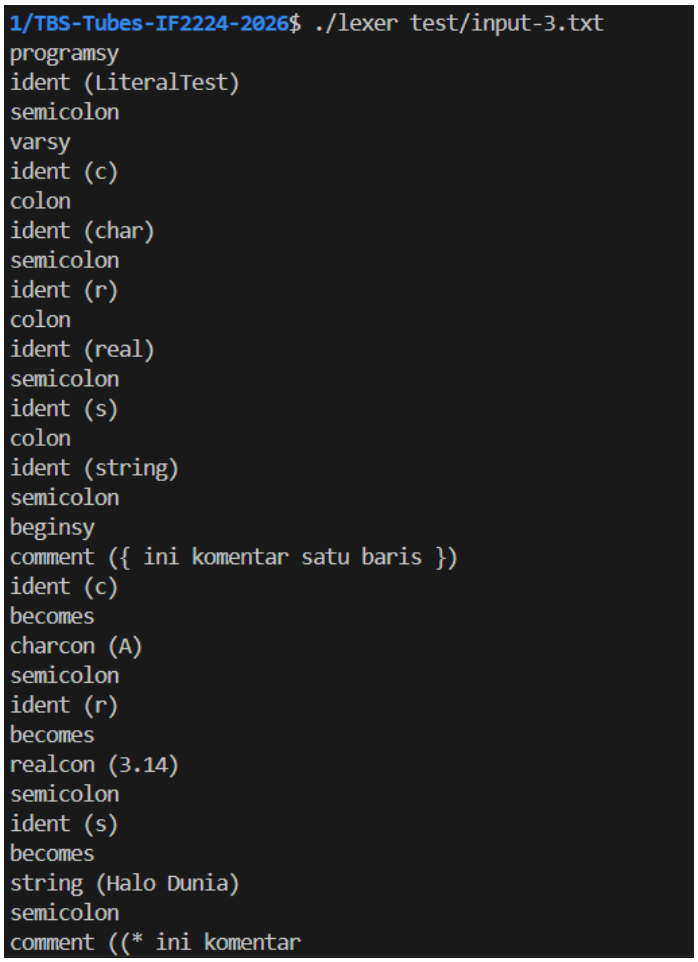
Tujuan	Menguji pengenalan literal string, charcon, realcon, serta comment, juga memastikan comment dimasukkan sebagai token

Input	`program LiteralTest;`
	`var`
	`  c: char;`
	`  r: real;`
	`  s: string;`
	`begin`
	`  { ini komentar satu baris }`
	`  c := 'A';`
	`  r := 3.14;`
	`  s := 'Halo Dunia';`
	`  (* ini komentar`
	`  multi baris *)`
	`  c := '''';`
	`end.`

Expected Output	`programsy`
	`ident (LiteralTest)`
	`semicolon`
	`varsy`
	`ident (c)`
	`colon`
	`ident (char)`

	semicolon ident (r) colon ident (real) semicolon ident (s) colon ident (string) semicolon beginsy comment (ini komentar satu baris) ident (c) becomes charcon (A) semicolon ident (r) becomes realcon (3.14) semicolon ident (s) becomes string (Halo Dunia) semicolon comment (ini komentar multi baris) ident (c) becomes charcon (') semicolon endsy period
Output	ident (LiteralTest) semicolon varsy ident (c) colon ident (char) semicolon ident (r) colon

	ident (real) semicolon ident (s) colon ident (string) semicolon beginsy comment ({ ini komentar satu baris }) ident (c) becomes charcon ('A') semicolon ident (r) becomes realcon (3.14) semicolon ident (s) becomes string ('Halo Dunia') semicolon comment ((* ini komentar multi baris *)) ident (c) becomes charcon (') semicolon endsy period

Bukti	

1.2.4. Test Case 4

Tujuan	Menguji bahwa keyword bersifat case-insensitive (`NOT`, `AND`, `OR` `DIV`, `MOD`, `IF`, `THEN`, dsb), dan bahwa identifier yang mengandung nama keyword sebagai prefiks tetap dikenali sebagai `ident`, bukan keyword
Input	`program CaseTest;` `var`

	`  android, notvalid, modulo: integer;`
	`begin`
	`  if NOT (android == 0) AND (notvalid <> 1) OR (modulo > 2) THEN`
	`  android := 10 Div 2;`
	`  android := android Mod 3;`
	`end.`

Expected Output	`programsy`
	`ident (CaseTest)`
	`semicolon`
	`varsy`
	`ident (android)`
	`comma`
	`ident (notvalid)`
	`comma`
	`ident (modulo)`
	`colon`
	`ident (integer)`
	`semicolon`
	`beginsy`
	`ifsy`
	`notsy`
	`lparent`
	`ident (android)`
	`eql`
	`intcon (0)`
	`rparent`
	`andsy`
	`lparent`
	`ident (notvalid)`
	`neq`
	`intcon (1)`
	`rparent`
	`orsy`
	`lparent`
	`ident (modulo)`
	`gtr`

	intcon (2) rparent thensy ident (android) becomes intcon (10) idiv intcon (2) semicolon ident (android) becomes ident (android) imod intcon (3) semicolon endsy period
Output	programsy ident (CaseTest) semicolon varsy ident (android) comma ident (notvalid) comma ident (modulo) colon ident (integer) semicolon beginsy ifsy notsy lparent ident (android) eql intcon (0) rparent andsy lparent

	ident (notvalid) neq intcon (1) rparent orsy lparent ident (modulo) gtr intcon (2) rparent thensy ident (android) becomes intcon (10) idiv intcon (2) semicolon ident (android) becomes ident (android) imod intcon (3) semicolon endsy period

Bukti	1/TBS-Tubes-IF2224-2026$ ./lexer test/input-4.txt programsy ident (CaseTest) semicolon varsy ident (android) comma ident (notvalid) comma ident (modulo) colon ident (integer) semicolon beginsy ifsy notsy lparent ident (android) eql intcon (0) rparent andsy lparent ident (notvalid) neq intcon (1) rparent orsy lparent ident (modulo) gtr intcon (2) rparent thensy ident (android) becomes intcon (10) idiv intcon (2) semicolon ident (android) becomes ident (android) imod intcon (3) semicolon endsy period

1.2.5. Test Case 5

Tujuan	Menguji bahwa lexer tidak crash saat menemukan karakter/token tidak valid, dan tetap melanjutkan

	tokenisasi hingga akhir file. Token yang tidak dikenali harus dikeluarkan sebagai `unknown`.

Input	`program ErrorTest;`
	`var`
	`  x: integer;`
	`begin`
	`  x := 5;`
	`  x := x + @;`
	`  x := #10;`
	`  y := 3 $ 2;`
	`end.`

Expected Output	`programsy`
	`ident (ErrorTest)`
	`semicolon`
	`varsy`
	`ident (x)`
	`colon`
	`ident (integer)`
	`semicolon`
	`beginsy`
	`ident (x)`
	`becomes`
	`intcon (5)`
	`semicolon`
	`ident (x)`
	`becomes`
	`ident (x)`
	`plus`
	`unknown (@)`
	`semicolon`
	`ident (x)`
	`becomes`
	`unknown (#)`
	`intcon (10)`
	`semicolon`
	`ident (y)`
	`becomes`
	`intcon (3)`
	`unknown ($)`

	intcon (2) semicolon endsy period
Output	programsy ident (ErrorTest) semicolon varsy ident (x) colon ident (integer) semicolon beginsy ident (x) becomes intcon (5) semicolon ident (x) becomes ident (x) plus unknown (@) semicolon ident (x) becomes unknown (#) intcon (10) semicolon ident (y) becomes intcon (3) unknown ($) intcon (2) semicolon endsy period

Bukti	1/TBS-Tubes-IF2224-2026$ ./lexer test/input-5.txt programsy ident (ErrorTest) semicolon varsy ident (x) colon ident (integer) semicolon beginsy ident (x) becomes intcon (5) semicolon ident (x) becomes ident (x) plus unknown (@) semicolon ident (x) becomes unknown (#) intcon (10) semicolon ident (y) becomes intcon (3) unknown ($) intcon (2) semicolon endsy period

3.2 Analisis Hasil Pengujian

3.2.1 Analisis Hasil Pengujian Test Case 1

Test Case 1 bertujuan menguji pengenalan token-token dasar yang mencakup keyword (program, var, begin, end), identifier, operator assignment (:=), literal integer, simbol titik dua (:), titik koma (;), dan period (.). Output yang dihasilkan program sepenuhnya sesuai dengan expected output, membuktikan bahwa :

- Mesin DFA berhasil menavigasi jalur keyword untuk mengenali PROGRAMSY, VARSY, BEGINSY, dan ENDSY.
- Identifier seperti Halo dan x yang bukan merupakan keyword dikenali dengan benar sebagai token IDENT.
- Operator dua karakter := (BECOMES) berhasil dikenali.
- Literal integer 42 berhasil dikenali sebagai INTCON(42).
- Whitespace dan newline diabaikan dengan benar tanpa menghasilkan token.

3.2.2 Analisis Hasil Pengujian Test Case 2

Test Case 2 menguji cakupan menyeluruh terhadap operator aritmatika (+, -, *, /, div, mod) dan seluruh operator relasional (==, <>, >, >=, <, <=). Output program identik dengan expected output, yang membuktikan :

- Operator aritmatika satu karakter (+, -, *, /) dikenali langsung sebagai PLUS, MINUS, TIMES, dan RDIV melalui transisi tunggal dari state 0.
- Keyword operator div dan MOD berhasil dikenali sebagai IDIV dan IMOD. Kasus MOD (huruf kapital) membuktikan bahwa keyword map berjalan secara case-insensitive, karena lexeme dinormalisasi ke huruf kecil sebelum pencarian.
- Operator relasional dua karakter (>=, <=, <>, ==) berhasil dikenali. Untuk < khususnya, tiga kemungkinan (LSS, LEQ, NEQ) dibedakan dengan benar.
- Operator relasional satu karakter > juga berhasil dibedakan dari >= (GTR vs GEQ).

3.2.3 Analisis Hasil Pengujian Test Case 3

Test Case 3 menguji fitur-fitur leksikal yang lebih kompleks: char literal, string literal, real constant, dan komentar dalam dua format ({ } dan (* *)). Secara keseluruhan output program menunjukkan kesesuaian fungsional yang tinggi dengan expected output, dengan satu perbedaan kosmetik pada format pencetakan:

- Realcon 3.14 berhasil dikenali dengan benar. Mesin DFA di state n1 mendeteksi titik yang diikuti digit sehingga berpindah ke state n2 dan menerbitkan REALCON(3.14).
- Char literal 'A' berhasil dikenali sebagai CHARCON. Implementasi membedakan char literal (tepat satu karakter di antara tanda kutip) dari string literal (lebih dari satu karakter).
- String literal 'Halo Dunia' (berisi spasi, lebih dari satu karakter) berhasil dikenali sebagai STRING.
- Komentar format { } dikenali sebagai token COMMENT. Isi badan komentar turut disertakan sebagai value token.
- Komentar format (* *) multi-baris juga berhasil dikenali sebagai token COMMENT.
- Perbedaan minor: output program mencetak token dengan tanda kutip pembungkus (misalnya charcon('A') dan string('Halo Dunia')), sementara expected output mencantumkan tanpa tanda kutip tambahan (charcon(A)). Perbedaan ini bersifat kosmetik pada format pencetakan dan tidak mempengaruhi kebenaran logika tokenisasi.

- Char literal escape '" (tanda kutip ganda di dalam char literal) berhasil ditangani dengan benar.

3.2.4 Analisis Hasil Pengujian Test Case 4

Test Case 4 adalah uji komprehensif untuk dua properti penting: (1) case-insensitivity keyword, dan (2) kemampuan lexer membedakan identifier yang kebetulan diawali dengan nama keyword (misalnya android mengandung and, notvalid mengandung not, modulo mengandung mod). Output program identik dengan expected output, membuktikan :

- NOT, AND, OR, DIV, MOD ditulis dengan huruf kapital dalam input, namun tetap dikenali sebagai keyword NOTSY, ANDSY, ORSY, IDIV, IMOD. Hal ini membuktikan normalisasi lowercase pada lexeme sebelum pencarian di keyword map berjalan dengan benar.
- Identifier android tidak salah dipotong menjadi and + roid. Mesin DFA mengakumulasikan seluruh karakter alfanumerik dalam self-loop state l1 hingga delimiter ditemukan, sehingga seluruh lexeme android diperiksa dan tidak cocok dengan keyword manapun, menghasilkan IDENT(android) yang benar.
- Demikian pula notvalid dikenali sebagai IDENT(notvalid) dan modulo sebagai IDENT(modulo), bukan sebagai keyword.
- Keyword THEN (huruf kapital) dikenali sebagai THENSY, mengonfirmasi case-insensitivity berlaku untuk semua keyword.

3.2.5 Analisis Hasil Pengujian Test Case 5

Test Case 5 menguji ketahanan lexer ketika menghadapi karakter-karakter tidak valid (@, #, $). Ini adalah uji kritis yang memastikan lexer tidak crash atau berhenti di tengah proses, melainkan terus melanjutkan tokenisasi hingga EOF. Output program identik dengan expected output, membuktikan :

- Karakter @ setelah operator + dikenali sebagai UNKNOWN(@). Lexer kemudian melanjutkan dari state 0 dan berhasil mengenali ; berikutnya sebagai SEMICOLON.
- Karakter # sebelum 10 dikenali sebagai UNKNOWN(#), kemudian 10 diproses secara mandiri dan menghasilkan INTCON(10) yang benar.
- Karakter $ diapit dua integer dikenali sebagai UNKNOWN($), dan kedua integer di kiri-kanannya berhasil dikenali sebagai INTCON.

- Identifier y yang belum dideklarasikan tetap dikenali sebagai IDENT(y) tanpa error, karena pengecekan deklarasi bukan merupakan tugas lexer, melainkan tugas semantic analyzer.
- Program menyelesaikan tokenisasi seluruh file hingga ENDSY dan PERIOD meski terdapat beberapa karakter error, membuktikan mekanisme error recovery berjalan dengan baik.

BAB IV
Kesimpulan dan Saran

4.1 Kesimpulan

Dalam proyek ini, kelompok kami telah berhasil mengimplementasikan *lexical analyzer* untuk bahasa Arion dengan menggunakan mesin *Deterministic Finite Automata* (DFA) berbasis C++. Kami memilih model DFA karena terbukti sangat efisien dan memadai dalam menangani seluruh pola token bahasa Arion yang memang termasuk dalam kategori bahasa reguler. Dengan model ini, kami tidak perlu menggunakan mesin komputasi yang lebih kompleks seperti PDA atau *Turing Machine*, karena DFA sudah cukup kuat untuk memproses setiap aliran karakter masukan dan mengubahnya menjadi token-token yang akurat sesuai spesifikasi.

Seluruh kategori token yang dibutuhkan telah kami implementasikan secara menyeluruh, mulai dari 25 kata kunci, *identifier*, berbagai literal, hingga operator dan komentar. Kelompok kami juga memberikan perhatian khusus pada logika resolusi antara *keyword* dan *identifier*. Dengan menerapkan sistem *map* yang bersifat *case-insensitive*, kami berhasil memastikan bahwa *identifier* yang diawali nama kata kunci tidak akan salah terklasifikasi, serta kata kunci tetap bisa dikenali meski ditulis dengan kombinasi huruf besar-kecil yang berbeda. Selain itu, kami menyertakan mekanisme *error recovery* yang efektif, di mana karakter tidak valid akan ditandai sebagai token UNKNOWN sehingga proses tokenisasi tetap bisa berlanjut sampai selesai tanpa mengalami *crash*.

Berdasarkan hasil pengujian terhadap lima *test case* yang tersedia, kelompok kami mencapai tingkat kelulusan yang sangat memuaskan karena seluruh output sudah sesuai dengan *expected output*. Kami memang menemukan sedikit perbedaan pada Test Case 3, namun hal tersebut hanya bersifat kosmetik pada format pencetakan tanda kutip di bagian CHARCON dan STRING, sehingga tidak memengaruhi kebenaran logika tokenisasi secara keseluruhan. Secara umum, kelompok kami menyimpulkan bahwa implementasi DFA ini sudah bekerja dengan sangat baik dan stabil dalam menjalankan tugasnya sebagai tahap awal dari proses kompilasi bahasa Arion.

4.2 Saran

Berdasarkan hasil pengerjaan Milestone 1 ini, kami menyarankan pengembangan lebih lanjut pada sistem pelaporan kesalahan (*error reporting*). Saat ini, *lexer* yang kami buat baru memberikan label UNKNOWN tanpa menyertakan lokasi spesifik kesalahan. Ke depannya, penambahan informasi baris (*line number*) dan kolom (*column number*) pada setiap token akan sangat membantu pengguna dalam proses *debugging* kode sumber Arion. Selain itu, kami juga melihat potensi untuk mengubah pendekatan variabel status eksplisit yang saat ini digunakan menjadi *Table-Driven DFA*. Dengan memisahkan logika mesin dari data transisi, struktur kode akan menjadi lebih modular, lebih bersih, dan jauh

lebih mudah jika ingin dilakukan penambahan pola token baru di masa mendatang tanpa harus merombak banyak logika program.

Saran selanjutnya berkaitan dengan persiapan integrasi ke tahap *parsing* pada Milestone 2 nanti. Kami menyarankan agar struktur data aliran token (*token stream*) yang dihasilkan saat ini dioptimasi lebih lanjut agar mendukung mekanisme *look-ahead* yang lebih efisien bagi *parser*. Hal ini penting untuk memastikan transisi antar tahap kompilasi berjalan mulus dan stabil. Selain itu, meskipun saat ini komentar bersarang (*nested comments*) dianggap sebagai kesalahan sesuai spesifikasi Arion, mempertimbangkan dukungan fitur tersebut di masa depan bisa menjadi nilai tambah bagi fleksibilitas bahasa ini. Walaupun DFA memiliki keterbatasan dalam menangani struktur bersarang, fitur ini tetap bisa diwujudkan dengan sedikit modifikasi tambahan pada mesin automata agar bahasa Arion menjadi lebih *user-friendly*.

Lampiran

Link Repository

https://github.com/tmthyberd/TBS-Tubes-IF2224-2026

Link Diagram Transisi

https://drive.google.com/file/d/1vgxTdVBaD0XwNf9s9x3pdcey6daj4Fyn/view?usp=sharing

Pembagian Tugas

NIM	Deskripsi Tugas	Persentase
13524012	Membuat DFA, membuat kerangka program, membuat laporan bagian 3.2, membuat program utama	25%
13524076	Membuat DFA, merevisi program, membuat laporan bagian 1.4, 2.1, merapihkan laporan	25%
13524078	Membuat DFA, membuat template laporan, laporan bagian 1.3, 2.1, Bab 3, dan Bab 4	25%
13524092	Membuat DFA, merevisi DFA berdasarkan QnA, pewarnaan DFA, membuat laporan bagian 1.1 dan 1.2	25%

Referensi

"Compiler Design - Lexical Analysis"
https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm

"Introduction to Automata Theory, Languages, and Computation 3rd Edition"
https://cdn-edunex.itb.ac.id/29161-Formal-Language-Theory-and-Automata/1629640939613_Introduction-to-Automata-Theory,-Languages,-and-Computation-Edition-3.pdf

"Compilers - Principles, Techniques, and Tools (2006)"
https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20(2006).pdf

"An Introduction to Makefiles"
https://www.gnu.org/software/make/manual/html_node/Introduction.html